

Μεταγλωττιστές 2025-26

Λογικοί τελεστές και τελεστές σύγκρισης
(για χρήση σε εντολές if/while)

Ο στόχος

- Προσθήκη λογικών τελεστών
 - `and`, `or`, `not`
- Και τελεστών σύγκρισης
 - `==`, `!=`, `>`, `>=`, `<`, `<=`
- Για χρήση σε **συνθήκες** εντολών `if` και `while`
 - *If `lexpr` block-statement ή while `lexpr` block-statement*
 - Οι συνθήκες `lexpr` είναι **λογικές εκφράσεις** που όταν υπολογιστούν δίνουν τελικές τιμές αληθείς ή ψευδείς
- Συνδυασμοί **λογικών** εκφράσεων με λογικούς τελεστές
 - *`lexpr and lexpr`, `lexpr or lexpr`, `not lexpr`*
- Και **αριθμητικών** εκφράσεων με τελεστές σύγκρισης
 - π.χ. *`aexpr == aexpr`*

Οι νέοι τελεστές

Τελεστής (υψηλότερη προτεραιότητα πρώτα)	Σημείωση
<code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code>	δεν θα υποστηρίζονται «αλυσίδες» σύγκρισης όπως στην Python π.χ. <code>1 < x < 8</code>
<code>not</code>	
<code>and</code>	“short-circuiting”
<code>or</code>	“short-circuiting”

- Προσοχή: οι αριθμητικοί τελεστές έχουν **ακόμα μεγαλύτερη προτεραιότητα**
- Π.χ. το `a*85 >= 23 + b*0.1` ισοδυναμεί με `(a*85) >= (23 + b*0.1)`

Σχεδιαστικές αποφάσεις

- Κατασκευή γραμματικής
 - Θα πρέπει να είναι **LL(1)** ή «**LL(1)** με **hack**»
- Αναπαράσταση λογικών αποτελεσμάτων (true/false)
 - Στη γλώσσα μας υπάρχουν αυτή τη στιγμή μόνο **floats**
- AST και λειτουργία λογικών τελεστών
 - Θα πρέπει να υπακούν στη λογική του “**short-circuiting**” όπως στις «κανονικές» γλώσσες προγραμματισμού

Γραμματική λογικών εκφράσεων

- Μια πρώτη απόπειρα
 - 4 επίπεδα προτεραιότητας: or/and/not/σύγκριση+παρενθέσεις
 - Θεωρητικά τα and και or είναι αριστερά/δεξιά προσεταιριστικά
 - Προς το παρόν κρατάμε την «επίσημη» LL(1) μορφή (δεξιά προσεταιριστική)

```
Lexpr      → Lterm Lterm_tail  
Lterm_tail → or Lterm Lterm_tail | ε
```

Γραμματική λογικών εκφράσεων

- Μια πρώτη απόπειρα
 - 4 επίπεδα προτεραιότητας: or/and/not/σύγκριση+παρενθέσεις
 - Θεωρητικά τα and και or είναι αριστερά/δεξιά προσεταιριστικά
 - Προς το παρόν κρατάμε την «επίσημη» LL(1) μορφή (δεξιά προσεταιριστική)

```
Lexpr      → Lterm Lterm_tail  
Lterm_tail → or Lterm Lterm_tail | ε  
  
Lterm      → Lfactor Lfactor_tail  
Lfactor_tail → and Lfactor Lfactor_tail | ε
```

Γραμματική λογικών εκφράσεων

- Μια πρώτη απόπειρα
 - 4 επίπεδα προτεραιότητας: or/and/not/σύγκριση+παρενθέσεις
 - Θεωρητικά τα and και or είναι αριστερά/δεξιά προσεταιριστικά
 - Προς το παρόν κρατάμε την «επίσημη» LL(1) μορφή (δεξιά προσεταιριστική)

<code>Lexpr</code>	<code>→ Lterm Lterm_tail</code>
<code>Lterm_tail</code>	<code>→ or Lterm Lterm_tail ε</code>
<code>Lterm</code>	<code>→ Lfactor Lfactor_tail</code>
<code>Lfactor_tail</code>	<code>→ and Lfactor Lfactor_tail ε</code>
<code>Lfactor</code>	<code>→ not Lfactor Latom</code>

Γραμματική λογικών εκφράσεων

- Μια πρώτη απόπειρα
 - 4 επίπεδα προτεραιότητας: or/and/not/σύγκριση+παρενθέσεις
 - Θεωρητικά τα and και or είναι αριστερά/δεξιά προσεταιριστικά
 - Προς το παρόν κρατάμε την «επίσημη» LL(1) μορφή (δεξιά προσεταιριστική)

<code>Lexpr</code>	$\rightarrow$ <code>Lterm</code> <code>Lterm_tail</code>
<code>Lterm_tail</code>	$\rightarrow$ <code>or</code> <code>Lterm</code> <code>Lterm_tail</code> ϵ
<code>Lterm</code>	$\rightarrow$ <code>Lfactor</code> <code>Lfactor_tail</code>
<code>Lfactor_tail</code>	$\rightarrow$ <code>and</code> <code>Lfactor</code> <code>Lfactor_tail</code> ϵ
<code>Lfactor</code>	$\rightarrow$ <code>not</code> <code>Lfactor</code> <code>Latom</code>
<code>Latom</code>	$\rightarrow$ (<code>Lexpr</code>) <code>Expr</code> <code>Relop</code> <code>Expr</code>
<code>Relop</code>	$\rightarrow$ <code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code>

- `Expr` είναι οι αριθμητικές εκφράσεις
 - Τις έχουμε ήδη υλοποιήσει

Είναι η νέα γραμματική LL(1);

- Πρέπει να ελέγξουμε τη **συνολική** γραμματική
 - Λογικές και Αριθμητικές εκφράσεις

Lexpr	→ ...
...	→ ...
Latom	→ (Lexpr) Expr Relop Expr
...	→ ...
Expr	→ Term ...
...	→ ...
Factor	→ (Expr) id number

Σύγκρουση FIRST sets

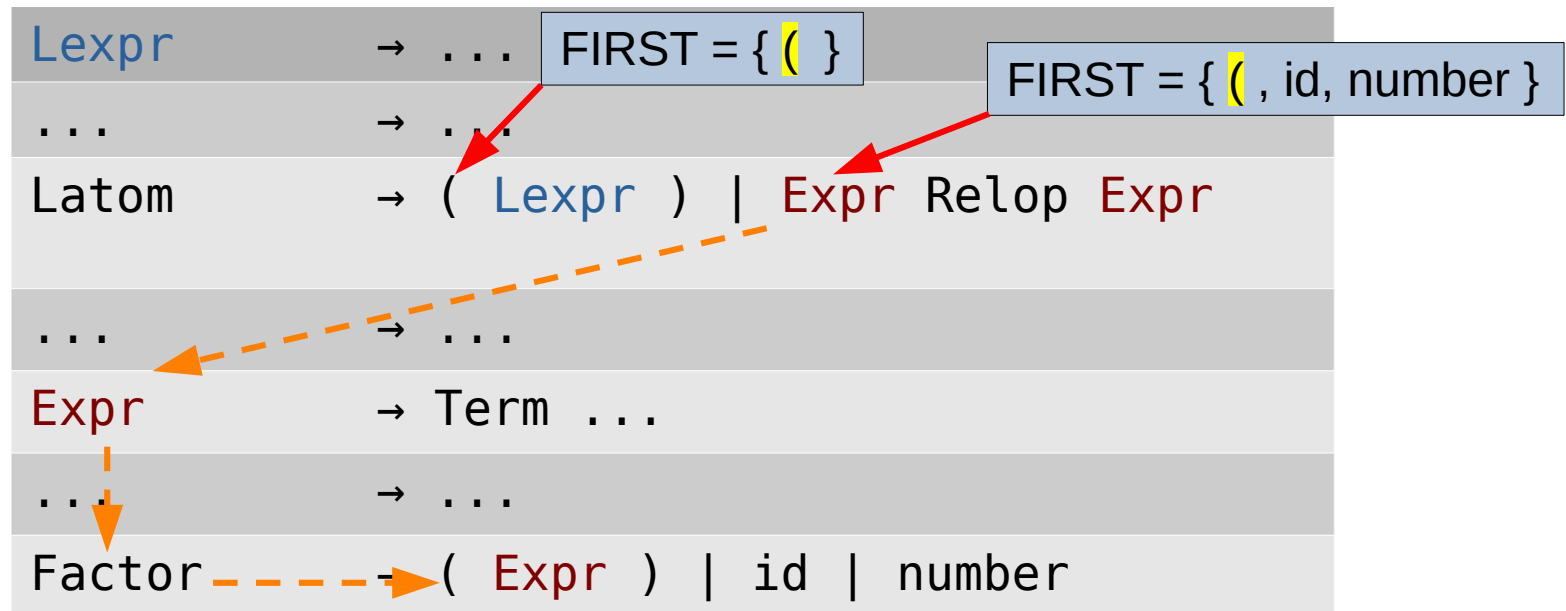
- Πρέπει να ελέγξουμε τη **συνολική** γραμματική
 - Λογικές και Αριθμητικές εκφράσεις

Lexpr	→ ...
...	→ ...
Latom	→ (Lexpr) Expr Relop Expr
...	→ ...
Expr	→ Term ...
...	→ ...
Factor	→ (Expr) id number

```
graph TD; Latom["Latom → ( Lexpr ) | Expr Relop Expr"]; Expr1["Expr → Term ..."]; Factor["Factor → ( Expr ) | id | number"]; Expr1 -.-> Latom; Expr1 -.-> Factor; Factor -.-> Expr1;
```

Σύγκρουση FIRST sets

- Πρακτικά: όταν είμαστε στο Latom και το επόμενο token είναι το (τι διαλέγουμε;



- Π.χ. ((((. . . . (((5 + ...
 - Ανοίγει παρένθεση λογικής ή αριθμητικής έκφρασης;
 - Δεν μπορούμε με 1 lookahead token (ή γενικότερα με k tokens) να αποφασίσουμε!
 - Αν αποφασίσουμε λάθος δεν μπορούμε να αλλάξουμε επιλογή...
 - Απαιτείται άλλη μέθοδος ανάλυσης ή backtracking

Εναλλακτικά: ενιαίος τύπος εκφράσεων

- Στη γραμματική ένα και μοναδικό **Expr** παντού
- Αριθμητική αναπαράσταση του true και false
 - ίσο με 0.0 → false
 - διάφορο του 0.0 → true
 - Όπως και σε πραγματικές γλώσσες προγραμματισμού (π.χ. η C)
- Χρήση **ανάλογα με τη θέση** που βρίσκεται η έκφραση
 - Ως μέρος αριθμητικής πράξης
 - Ως μέρος λογικής συνθήκης

Η γραμματική τώρα

```
Stmt_list  → Stmt Stmt_list | ε
Stmt       → id = Expr | print Expr
Expr       → Term (Addop Term)*
Term       → Factor (Multop Factor)*
Factor     → (Expr) | id | number
Addop      → + | -
Multop     → * | /
```

- Οι νέοι κανόνες για τις λογικές εκφράσεις θα πρέπει να **προηγηθούν** εφόσον οι νέοι τελεστές έχουν **χαμηλότερη προτεραιότητα** από τους αριθμητικούς

Η νέα γραμματική

Stmt_list → Stmt Stmt_list | ε

Stmt → id = Lexpr | print Lexpr

Lexpr → Lterm Lterm_tail

Lterm_tail → or Lterm Lterm_tail | ε

Lterm → Lfactor Lfactor_tail

Lfactor_tail → and Lfactor Lfactor_tail | ε

Lfactor → not Lfactor | Expr Rest

Rest → Relop Expr | ε

Expr → Term (Addop Term)*

Term → Factor (Multop Factor)*

Factor → (Lexpr) | id | number

Addop → + | -

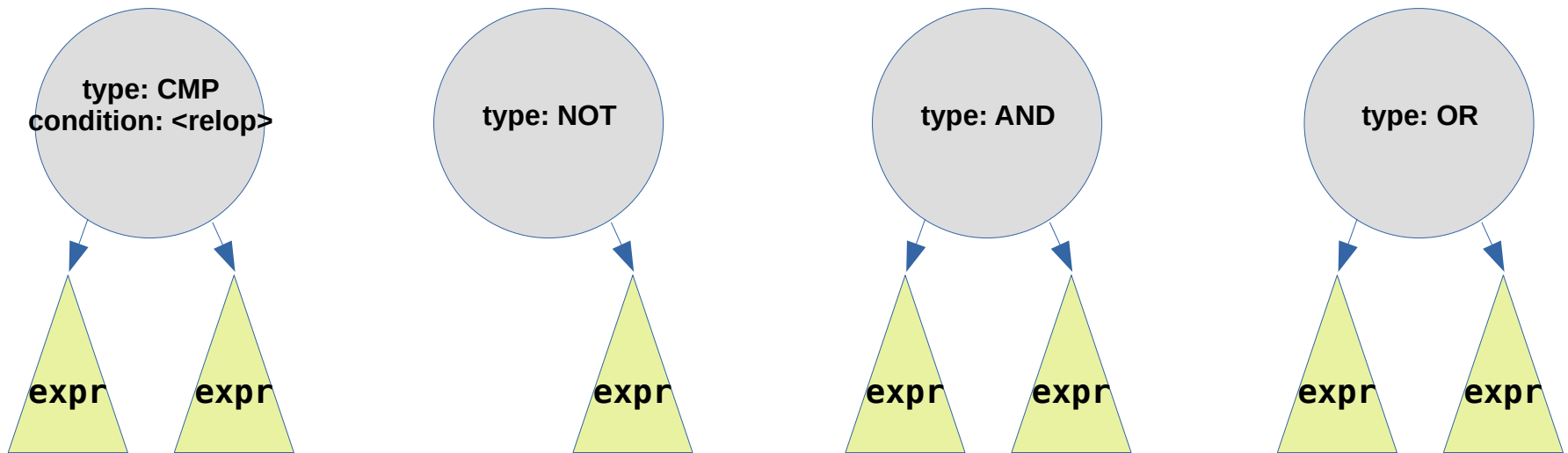
Multop → * | /

Relop → == | != | > | >= | < | <=

FIRST & FOLLOW sets

nonterminal	first set	follow set
Stmt_list	id print	#
Stmt	id print	
Lexpr	not (id number	
Lterm_tail	or	) id print #
Lterm	not (id number	
Lfactor_tail	and	) or id print #
Lfactor	not (id number	
Rest	== != > >= < <=	) and or id print #
Expr	(id number	
Term	(id number	
Factor	(id number	
Multop	* /	
Addop	+ -	
Relop	== != > >= < <=	

Κατασκευή AST: οι νέοι κόμβοι

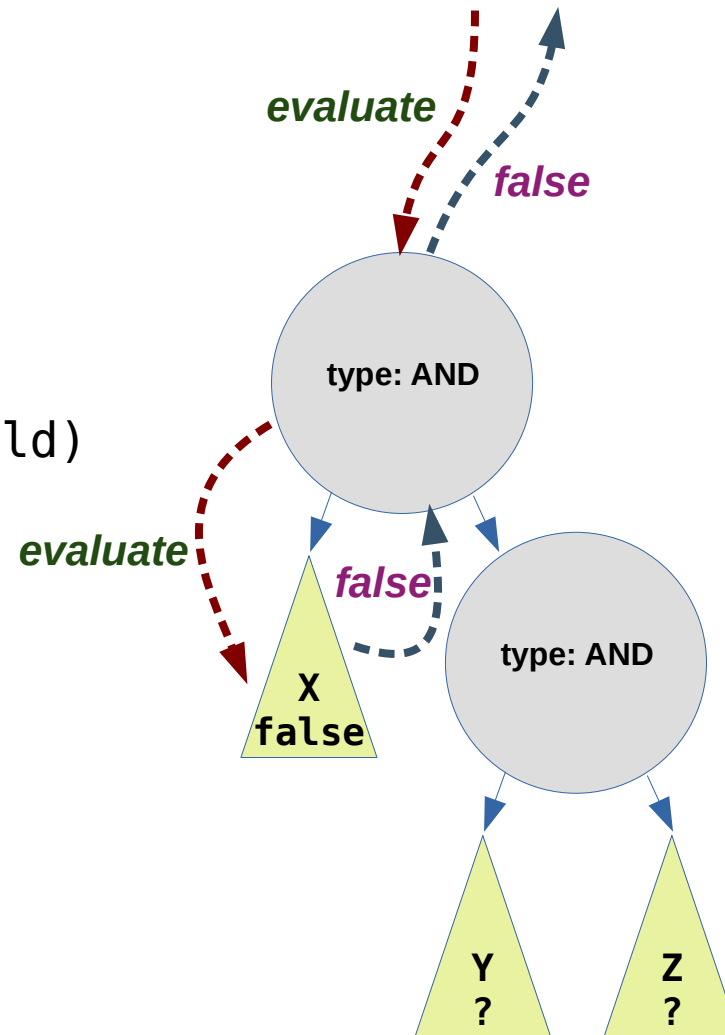


“Short-circuiting”

- Σε όλες τις γλώσσες προγραμματισμού
X and Y and Z and ...
 - Εάν το X είναι false **δεν υπολογίζονται** τα Y, Z ...
 - Μόνο εάν το X είναι true, τότε προχωράμε στον υπολογισμό του Y, κ.λ.π. προς τα δεξιά
 - Το ίδιο για το or: αν το X είναι true δεν **υπολογίζουμε** τα Y, Z...
 - Προκύπτει από τις ιδιότητες των and/or
- **Προσοχή:** δεν είναι (μόνο) θέμα βελτιστοποίησης
 - Ο υπολογισμός των Y, Z .. μπορεί να προκαλεί σφάλματα όταν το X είναι false
 - Για την ακρίβεια σε πολλές γλώσσες προγραμματισμού γίνεται ακριβώς αυτό
 - Ο προγραμματιστής **προστατεύει από την εκτέλεση ενός άκυρου Y** μέσω της κατάλληλης συνθήκης στο X

Η λειτουργία του short-circuiting

```
if type=='AND' {  
  a = evaluate(left_child)  
  if a is false  
    return false  
  else  
    return evaluate(right_child)  
}
```



Δοκιμαστικός κώδικας εισόδου

```
a = 2 + 7.55*44  
print a  
b = 3*(a-99.01)  
print b*0.23  
c = 5-3-2  
print c  
print c > a-1255 and not (b == 56*a or c != 0 or a>=b)
```